MAL • DATA PLATFORM • DELIVERABLE 1 OF 3

Customer 360 & CAR: Architecture & Design Rationale

What each of the three source systems actually knows, how identity resolution turns three partial views into one governed customer, and why every schema, tier, and refresh cadence in this platform is shaped the way it is.

DELIVERABLE 1 OF 3 — ARCHITECTURE DOCUMENT

Prepared For	Companion Documents	Status	Stack
Product, AI/ML, Compliance	#2 ERD #3 Identity SQL	Draft for review	AWS: S3, Glue, Redshift, RDS, Lake Formation

Contents

How to Read This Document

This is the first of three deliverables and it is deliberately scoped to *what* the platform does and *why* it is shaped the way it is, for an audience of data engineers who need to understand the design well enough to build against it, extend it, or challenge it. It explains the reasoning behind every major decision – not just the resulting schema – because a design that can't be justified out loud usually hasn't actually been thought through.

Two things live elsewhere on purpose: the complete field-level entity-relationship diagram is **Deliverable 2**, and the full, runnable SQL and pseudocode for identity resolution is **Deliverable 3**. This document references both and shows short illustrative fragments, but does not duplicate them – each deliverable earns its own focus rather than three overlapping copies of the same material.

Throughout, boxes marked **Why this way**call out the reasoning behind a decision explicitly, separated from the specification itself, so a reader can skim the what and still find the why when they need it.

1 The Data Landscape Today

Mal's customer data lives in three systems that were never asked to agree with each other, and each one is genuinely good at knowing a different, narrow slice of the truth. Before any schema or matching logic makes sense, it's worth looking at what each source actually contains – with realistic (fabricated) example records – because the differences between them are exactly what drives every downstream design choice.

1.1 CRM (Salesforce) – What Marketing and Support Know

Salesforce holds contact details, marketing consent, lead source, and support case history. It is the system of record for *intent* – what a person said they wanted, whether they opted into a campaign – but it has never verified anyone's identity. A phone number or email here is self-declared at sign-up, not confirmed by a regulated process.

Pseudo Example: Salesforce Contact

```
{
  "Id": "003Qy00000f8xYZIAY",
  "FirstName": "Amina",
  "LastName": "Al Suwaidi",
  "Email": "amina.s92@gmail.com",
  "Phone": "+971501234567",
  "LeadSource": "Instagram Ad",
  "HasOptedOutOfEmail": false,
  "MailingCountry": "AE",
  "CreatedDate": "2026-02-11T09:14:00Z",
  "LastModifiedDate": "2026-06-02T15:40:12Z"
}
```

Notice this contact has no national ID field at all – Salesforce was never built to hold one – and the email on file (a personal Gmail address) may not be the same email the person later verifies with the bank. That gap is normal, not an error, and the matching logic in Section 2 is built around expecting it.

1.2 Core Banking Platform – What Compliance Has Verified

This is the only one of the three systems that has been through a regulated KYC process. Every field here that matters for identity has been checked against a real document, which is exactly why it is treated as the trust anchor for the whole platform.

Pseudo Example: Core Banking Customer Record

```
{
  "cif_number": "CIF-000481932",
  "full_legal_name": "AMINA MOHAMED AL SUWAIDI",
  "date_of_birth": "1997-03-22",
  "national_id_number": "784-1997-XXXXXXX-1",
  "verified_mobile": "+971501234567",
  "verified_email": "amina.alsuwaidi@icloud.com",
  "kyc_status": "VERIFIED",
  "risk_rating": "LOW",
  "residency_country": "AE",
  "account_open_date": "2026-03-01",
  "product_holdings": ["CURRENT_ACCOUNT", "SAVINGS_MURABAHA"]
}
```

Two things worth noticing, because they show up again in Section 2: the legal name is stored in a different case and word order than the CRM contact (routine formatting drift, not a real disagreement), and the verified email is a completely different address than the one Salesforce has on file. If matching relied on email, these would look like two different people.

1.3 Mobile App Analytics (Amplitude) – What a Device Did, Before Anyone Confirmed Whose It Is

Amplitude knows behavior, not identity. Every event starts life attached to an anonymous device, and only becomes attributable to a real customer once an authenticated app session confirms which verified mobile number that device belongs to.

Pseudo Example: Amplitude Event, Before and After Login

```
// Before login: anonymous, unattached
{
  "user_id": null,
  "device_id": "d3f1-92ab-77cd-441e",
  "event_type": "session_start",
  "platform": "iOS",
  "app_version": "2.4.1",
  "event_time": "2026-06-10T22:03:44Z",
  "user_properties": {}
}

// After an authenticated session: device is bound, never before
{
  "user_id": null,
  "device_id": "d3f1-92ab-77cd-441e",
  "event_type": "feature_used",
  "event_properties": {"feature": "profit_payout_schedule"},
  "event_time": "2026-06-11T08:12:09Z",
  "user_properties": {"login_bound_mobile_e164": "+971501234567"}
}
```

> **WHY THIS WAY**: Amplitude is deliberately never allowed to originate an identity link on its own – a device is not a person, and inferring one from usage patterns alone (the phone is mostly used at night, therefore it's probably this customer) is exactly the kind of soft, unauditable guess that has no place deciding who a bank thinks someone is. The `login_bound_mobile_e164` field only ever

WHY THIS WAY (CONT.)

gets populated by an authenticated session event, which means Amplitude's contribution to identity resolution is always downstream of, and gated by, a real login.

1.4 Why None of These Three Can Be the Source of Truth Alone

Core banking is the most trustworthy but the narrowest – it only knows people who have actually opened an account, so a marketing lead who hasn't converted yet doesn't exist there at all. CRM knows about people earlier in the funnel but has verified nothing. Amplitude knows the richest behavioral detail but starts every device as a stranger. A usable Customer 360 has to combine all three without pretending any single one is complete on its own – which is precisely the job of identity resolution, covered next.

2 Identity Resolution Strategy

Identity resolution answers one question for the whole platform: given records from three systems that each mint their own IDs, which ones describe the same human being? Get this wrong in either direction – missing a real match, or worse, merging two different people – and every feature built on top of it inherits the mistake.

2.1 Matching Logic: A Trust-Ordered Waterfall

Matching runs as a waterfall of decreasing confidence. A record is checked against the strongest available signal first; if it matches, resolution stops there and weaker signals are never consulted. If it doesn't match – or the field doesn't exist for that record – it falls through to the next signal down.

Pass	Key	Basis	Confidence
1	National ID (tokenized)	Deterministic exact match	Very high
2	Verified mobile, E.164 (tokenized)	Deterministic exact match	High
3	Verified email (tokenized)	Deterministic exact match	Medium-high
4	Name similarity + date of birth + geography	Probabilistic, threshold-gated	Graded

Walking the Example Through the Waterfall

Take the three pseudo records from Section 1. The core banking record mints an ECID from its national ID token at Pass 1, since it has completed KYC. The Salesforce contact has no national ID to compare, so it drops to Pass 2 – and its phone number, `+971501234567`, matches the core record's verified mobile exactly. It attaches there, at Pass 2, *despite* the two records disagreeing on email and having slightly different name formatting. The Amplitude device, once it logs an authenticated session with `login_bound_mobile_e164` equal to that same number, attaches at the same pass. Notice that Pass 4 – the expensive, approximate, human-reviewable fallback – was never needed for this customer at all. That is the point of ordering the waterfall by confidence: the fuzzy, costly matching effort is reserved for the genuinely hard residual cases, not spent on every record.

WHY THIS WAY: If matching instead started with email, this customer would not resolve automatically at all – `amina.s92@gmail.com` and `amina.alsuwaidi@icloud.com` share nothing in common. Ordering by trustworthiness rather than by which field happens to be easiest to compare is what makes

WHY THIS WAY (CONT.)

the waterfall actually work on real, messy data instead of only working on the clean records used in a demo.

2.2 Probabilistic Matching: Technology, Logic, and Where It Actually Fits

Passes 1–3 resolve any customer who has a verified mobile, email, or national ID in common across systems – which, in practice, covers the overwhelming majority of real matches, because most CRM leads that matter eventually verify a phone number during onboarding. Pass 4 exists for the residual: CRM leads who never gave a phone number that matches, or gave one with a typo, and have to be matched on fuzzier evidence instead.

The Underlying Idea: Probabilistic Record Linkage

Pass 4 is not ”does the name look kind of similar” run informally – it follows a formal, decades-old framework called **Fellegi–Sunter probabilistic record linkage**. For each comparable field (name, date of birth, geography), the model estimates two probabilities: the *m-probability*, how likely that field agrees when the records really are the same person, and the *u-probability*, how likely it agrees purely by chance when they are not. Combining these across fields produces a single composite match weight, which is compared against two cutoffs to sort every candidate pair into one of three outcomes: confidently the same person, confidently different people, or genuinely ambiguous and worth a human’s attention. That three-way split is exactly the auto-merge / steward-review / no-merge behavior described in Section 2.1’s confidence bands – it isn’t an ad hoc rule, it’s the standard shape this kind of problem takes once it’s modeled properly.

Technology Choice

Option	How It Decides a Match	Effort to Stand Up	Fit for Mal, Q2
AWS Glue FindMatches	Managed ML classifier; a human labels a sample of pairs through a console UI, AWS trains and hosts the model, you score the rest as a Glue job	Low — no infrastructure to run, reads/writes the same Glue Catalog tables already used for staging	**Selected**
Zingg (open-source, Spark)	Same Fellegi-Sunter-style approach with active-learning labeling, but self-hosted and fully inspectable	Medium — deploy on Glue/EMR Spark, own the labeling loop yourself	Credible fallback if Compliance ever wants to see model internals FindMatches doesn't expose
Custom PySpark + `recordlinkage`/ `dedupe`	Hand-built similarity features and a self-trained classifier	High — you own blocking, feature engineering, training, and retraining	Not justified for Q2 given the time budget
Hand-rolled SQL/UDF threshold (Jaro-Winkler only)	Fixed similarity threshold, no learned weighting, no probability calibration	Lowest	Acceptable Day-30 stopgap only, before a labeled dataset exists

WHY THIS WAY: FindMatches wins for Q2 for a boring but decisive reason: it is the only option that adds zero new infrastructure to operate, given the AWS-native constraint the whole platform already commits to. It reads directly from the same Glue Catalog the deterministic waterfall already populates, and the labeling work needed to train it is a business process (stewards reviewing sample pairs), not a data-engineering build. Zingg stays on the shortlist specifically because "why did the model think these matched" is a question Compliance will eventually ask, and FindMatches answers it less transparently than an open, inspectable model would – worth revisiting once the platform has more than a Q2 launch to worry about.

Feasibility: This Is Not a Free Feature

Training FindMatches needs a labeled seed set – a few hundred to low thousands of pairs marked "same person" or "different people" by a human who can actually tell. For Mal, that seed set comes from two places: pairs already confirmed by the deterministic waterfall (free, generated automatically as a byproduct of Passes 1–3) and a small manually-reviewed sample of genuinely ambiguous CRM leads, which realistically costs a KYC operations steward a few days of focused review time. This is why probabilistic matching is scheduled for the Day 60 milestone rather than Day 30 in the execution plan – there needs to be enough deterministic-match history first to seed the labels, and pretending otherwise just means shipping an untrained, uncalibrated matcher.

At Mal's Year 1 scale, the pool that actually needs Pass 4 should be modest: most people who matter to the business eventually verify a phone number during onboarding, so the residual is realistically a low tens-of-thousands of CRM-only leads out of 500K customers – comfortably inside what a nightly batch Glue job

can score without special tuning.

Integration Into the Pipeline

The probabilistic pass consumes exactly the leftovers of the deterministic waterfall – the CRM and core-banking records that reached Section 2's Pass 3 and still found nothing. It scores candidate pairs and writes them to a staging table with a match probability attached; pairs above the upper cutoff are inserted into the identity crosswalk automatically with `match_method = PROBABILISTIC_FINDMATCHES`, pairs in the ambiguous middle band go to a steward review queue, and pairs below the lower cutoff are logged but never merged. Steward decisions on the review queue are periodically fed back in as new labeled examples, so the model's calibration keeps improving with real corrections rather than staying frozen at its Day 60 starting point.

Is an Agent – an LLM-Based Model – Applicable Here?

Deliberately, no, not for the merge decision itself. This is a calibrated classification problem, not a reasoning or generation problem, and a bank needs to be able to say *exactly* why two records were merged in terms a regulator will accept – "the model scored this pair 0.94 based on name similarity and an exact date-of-birth match" is defensible; "a language model judged these were probably the same person" is not, and introduces exactly the kind of unaccountable judgment call this design goes out of its way to avoid everywhere else. There is one narrow, genuinely useful place an LLM-based assistant could help: summarizing *why* a specific pair landed in the steward review queue, in plain language, to speed up a human's decision – an assistive triage tool, not a decision-maker, and explicitly out of scope for Q2.

2.3 Survivorship Rules: Field-Level, Not Record-Level

When multiple source records resolve to one ECID, the golden record is assembled attribute by attribute, using a declared priority per field family rather than picking one source record wholesale.

Field Family	Winner	Rationale
Legal name, DOB, national ID, KYC status	Core banking	The only source that has verified these fields
Mobile, email	Core banking, falling back to CRM if null	Core banking's copy is OTP-verified
Marketing preferences	CRM	CRM is the system of record for marketing intent, regardless of what core banking happens to hold
Behavioral attributes	Amplitude, but never promoted into the identity record	Behavior belongs in the CAR as a feature, not in the identity record as a fact about who someone is
Two authoritative sources disagree	Most-recent update wins, flagged for review	Avoids silently masking a real data-quality problem

2.4 Golden Record Lifecycle

Land and tokenize → match and cluster → survive and merge → publish as SCD Type 2 → steward review for anything low-confidence. New customers are bootstrapped from the KYC event at account opening, which is what mints an ECID in the first place – a marketing lead never gets one on its own. Merges and

splits are handled through the identity crosswalk table, not by rewriting the golden record directly, which is what makes a bad merge reversible rather than destructive: undoing it is a crosswalk correction and an SCD2 replay, not a rebuild from scratch.

3 Customer 360 Schema

The Customer 360 is organized in three layers, each normalized differently because each layer does a different job – raw data lands as-is for replayability, a conformed layer cleans and types each source independently, and the golden layer is a star schema with SCD2 history, because that is what makes point-in-time correctness and safe updates possible at once.

Layer	Store	Normalization	Purpose
Bronze	S3, source-shaped	None – as landed	Immutable, replayable audit base
Silver	S3 + Glue Catalog	3NF-aligned per source	Cleaned, typed, deduplicated, ready for matching
Gold	Redshift + RDS	Star schema, SCD2 dimensions	The Customer 360 that Product, Support, and BI query

Gold sits in two stores for two different access patterns, not out of redundancy: Redshift is the analytical engine every BI query and the nightly CAR build hits, while a narrow RDS (PostgreSQL) replica – kept in sync by that same golden-layer publish job – serves the sub-100ms single-customer lookups the mobile app backend and the support console need, which a columnar MPP warehouse is the wrong tool for. Nothing writes to the RDS copy directly; it is read-only downstream of the same SCD2 publish that feeds Redshift, so the two stores never have a chance to disagree about what ”current” means for a given customer.

The core entities are Customer (golden, SCD2), Identity Crosswalk, Account, Product (including the Sharia contract type – Murabaha, Mudarabah, Wakala – rather than an interest rate), Transaction, Consent, Device Identity, Behavioral Event, and Support Ticket. One customer owns many accounts, one account posts many transactions, one customer holds many consents and raises many tickets, and one customer links to many device identities. The full field-level schema for every one of these entities, with primary and foreign keys and every cardinality drawn out, is **Deliverable 2 – the ERD**.

> **WHY THIS WAY**: Household or family relationship modeling is useful for both personalization and fraud-ring detection, but it is deliberately left out of this schema. It needs its own resolution logic layered on top of individual identity resolution, and neither the Q2 personalization use case nor the initial credit/fraud models need it to function – adding it now would be solving a Year 2 problem before the Year 1 foundation is proven.

4 PII Access Tier Model

Access is modeled as four tiers, enforced as close to the data as possible – column- and row-level policy inside Redshift and Lake Formation, not an application convention a new script could quietly ignore.

Tier	Who	What They See	Enforcement
Tier 0	Any employee with warehouse access	Aggregates only – no row-level customer data	Pre-aggregated marts; no PII columns exist
Tier 1	AI/ML engineers, general analysts	Direct identifiers tokenized; stable, joinable pseudonyms	Redshift dynamic masking by role; Lake Formation column grants
Tier 2	Support, KYC ops, compliance analysts	Full PII, scoped to assigned case/customer	Row-level security scoped to case; real values granted only to these roles
Tier 3	Named individuals, Financial Crime, by approval	Raw identifiers, cross-tier re-identification	Time-boxed break-glass, dual approval, fully audit-logged

Two pseudonymization techniques serve two different jobs. **Deterministic tokenization** (keyed HMAC-SHA256, key in KMS) gives the AI team a stable pseudonym that still joins correctly across tables without ever exposing the real value. A separate **reversible token vault** (DynamoDB, KMS customer-managed key) exists only for genuine Tier 3 re-identification needs, and every lookup against it is itself an independently audited event.

> **WHY THIS WAY**: Consent gates what gets *computed*, not only what gets shown. A customer who withdraws personalization consent should not have personalization features silently calculated and merely hidden behind a mask – that wastes compute and leaves a standing risk that a future bug exposes data that should never have existed downstream at all. The nightly CAR job checks consent before populating gated columns and leaves them null rather than computed-and-masked.

5 CAR v1 Schema

The Customer Analytic Record is a wide, deliberately denormalized snapshot – one row per customer per day – that pre-joins and pre-aggregates everything a credit or fraud model would otherwise reconstruct from ten normalized tables at query time. This pattern, and the name, are both standard in banking data warehousing – the Customer Analytic Record (CAR) is an established industry term for exactly this kind of wide, pre-aggregated, one-row-per-customer table, distinct from any single vendor's proprietary model – and it has stayed standard because it turns "build a feature" into "add a column to a table that already refreshes on schedule."

Group	Examples	Gated by Consent?
Identity & status	ecid, kyc_status, risk_rating, residency_country	No
Balance & holdings	total_balance, utilization_ratio	No (legitimate interest: risk)
Transaction behavior	txn_count_30d, cross_border_txn_flag	No (legitimate interest: risk/fraud)
Engagement	session_count_7d, dominant_channel	**Yes** – personalization consent
Model-ready flags	schema_version, feature_freshness_hours	No

A single nightly Glue job builds the day's snapshot from the closing state of the golden marts plus that day's incremental facts, runs it through a data-quality gate (Glue Data Quality / Deequ-style row-count, null-rate, and range checks), and only then publishes. A failed gate holds the previous day's snapshot live and pages the on-call, rather than ever shipping a partial one. Refresh is daily, targeted to complete comfortably inside the 24-hour freshness commitment – a ceiling, not an ambition, since select fraud-relevant features move toward a minutes-scale refresh once the streaming layer (a later phase of this engagement) exists.

6 AI Team Data Interface

The AI team consumes from a dedicated AI zone – the CAR published as native Redshift tables on a separate serverless workgroup (so training queries never compete with product dashboards) and mirrored as partitioned S3 Parquet for direct SageMaker consumption. Every release is described by a versioned schema contract (Glue Schema Registry, mirrored as a reviewable file in the ETL repo): backward-compatible changes ship within a version, anything breaking ships as a new version with a deprecation window.

Commitment	Target
Freshness	Full prior-day snapshot published under 24 hours
Completeness	At least 99.5% of active customers present
Schema stability	No breaking change without a new version and a 30-day dual-publish window
Data quality	Automated gate blocks publish; last-known-good snapshot served instead
Access	Pseudonymized by default; re-identification only via audited token-vault lookup

7 30/60/90 Day Execution Plan

The ordering follows one hard dependency chain: identity resolution before the golden Customer 360 means anything, the golden 360 before the CAR can be built, and PII tiering enforced before the AI team gets any access at all.

Day	Activities
30	Ingestion for all three sources into bronze with tokenization on write; deterministic waterfall only; golden core tables (dim_customer SCD2, dim_account, identity_ crosswalk); PII tier framework live; consent schema wired into onboarding; CAR skeleton (identity, balance, transaction columns); AI team granted narrow Tier 1 read access
60	Probabilistic matching (FindMatches) added with its labeled seed set; full behavioral feature set in the CAR; automated data-quality gate; versioned schema contract published; PII access audit logging live; consent-withdrawal batch job
90	Performance and cost tuning at production volume; break-glass Tier 3 workflow automated; household/relationship modeling design begins; streaming layer design begins; first governance council review; DR drill

This Document's Companions

The complete, field-level Customer 360 and CAR schema – every entity, key, and cardinality – is drawn out in **Deliverable 2 (ERD)**. The full, runnable SQL and pseudocode implementing the deterministic waterfall, the probabilistic fallback, and the survivorship merge described in Section 2 is in **Deliverable 3 (Identity Resolution SQL & Pseudocode)**.